CMPE2550 – ICA07 [ASP] – Tug of War

In this exercise, you will develop Tug of War Assignment 07 to incorporate ASP Core functionality and brush up on some client-side skills such as AJAX calls and dynamic page updating. You may start by making Assignment 07 folder, then rename the elements appropriately.

Tug of War is a common game, a contest in which two teams pull at opposite ends of a rope until one drags the other over a central line. Create a new ASP Assignment07 folder. You will require at least 4 files (exact names not required):

index.html This file will be your landing page of your game.

TugofWar.js This page will process the player's actions from the index.html page.

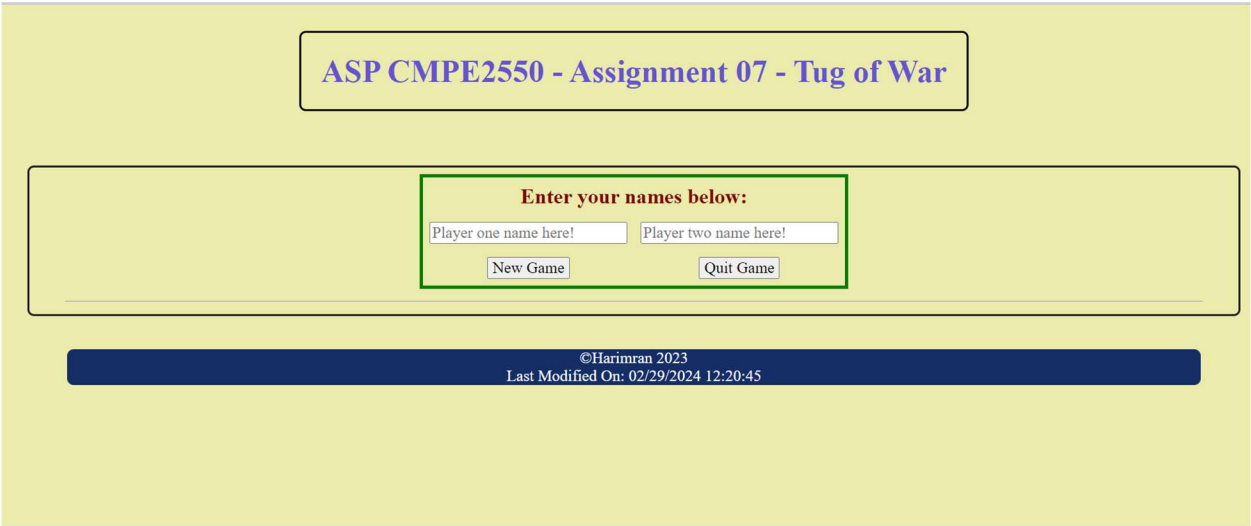
TugofWar.css This page will be used to contain both regular static classes, and classes that may be added and removed as required by the application logic (mostly for indicating types of user messages).

Web Service This will be a service file(s) for containing the "brains" of the application.

Or you can create an ASP.NET Core project and then create client-side files under a folder. It is demonstrated in the demo as well.

You may include other pages as you feel are necessary for organization and mobility purposes.

An example layout and very simple style has been shown, but it is not mandatory to replicate exactly. Feel free to be artistic as you please, but it is advised to focus on the functionality first. When the players first navigate to the page, a simple layout is seen such as that shown below:



The controls in the sample image are:

1) One Label for messaging the user

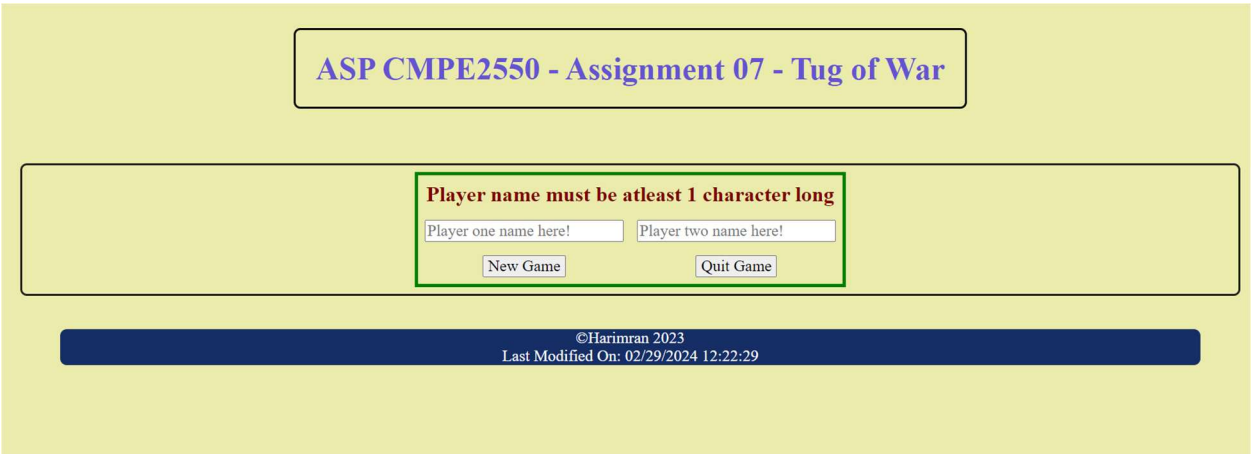
2) Two TextBoxes for the players' names

3) Two Buttons for new games (same players) and resetting the game (including players)

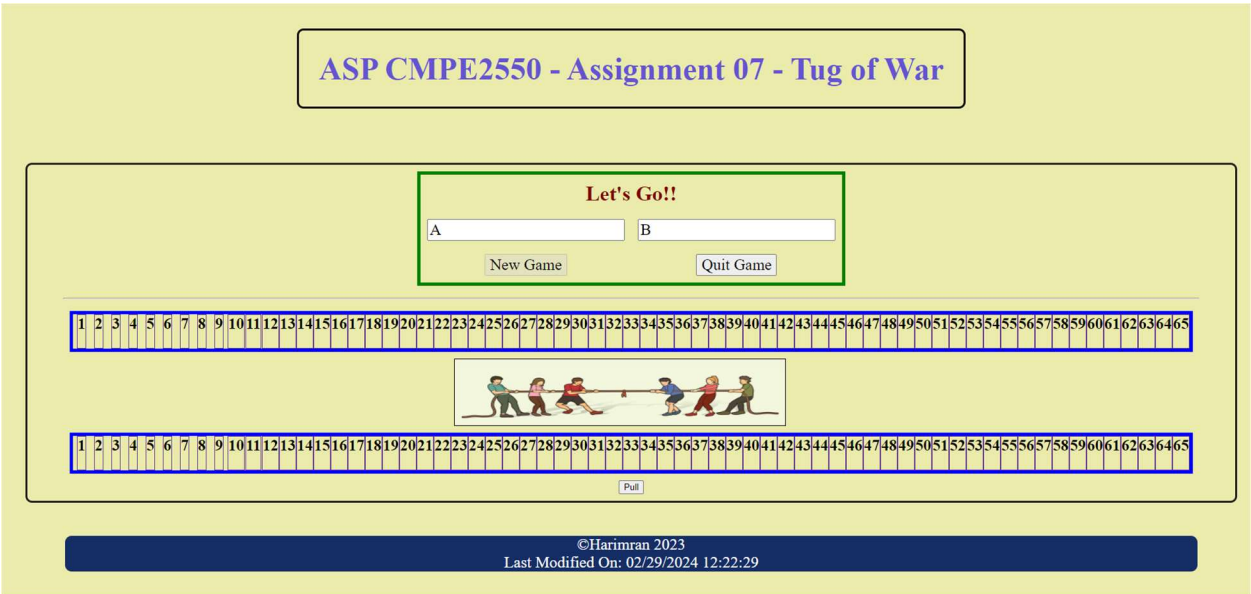
4) The blank gameboard (you could choose to show or hide this on New and Quit clicks)

This example is a minimum bar to strive for. As you complete the assignment, you may choose to add more to the basic requirements.

When the user presses the [New Game] Button, the submitted data will be cleaned and examined to ensure both names were provided. An appropriate error message should be displayed if either or both names are missing.

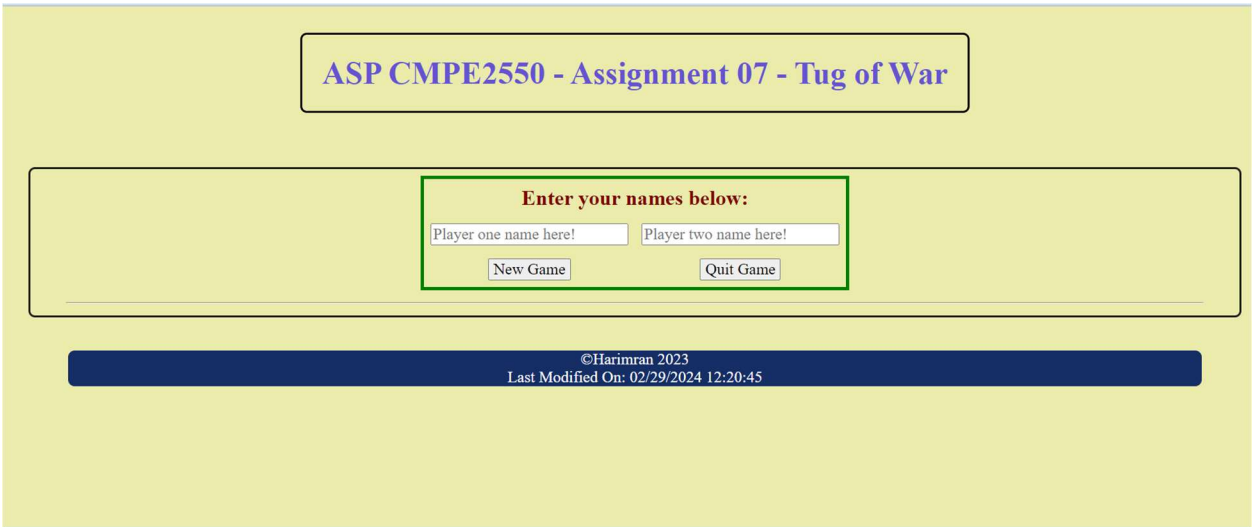


Once both names have been supplied, and game is ready to start, a message will be indicated in the status message label "Let's Go". Values should be stored on the server side so that the server may keep track of the game details. Make game stage visible along with Pull button to generate random forces for each player on the server side.



An AJAX call will be sent to the server when the page is loaded indicating that the server should initialize a new game. It is suggested to use separate server links to server or you can send an “action” to the server with all AJAX calls (perhaps “NewGame” here?).

If the [Quit Game] Button is pressed, the server data will be reset to default values. This should return you to the game state seen in the first image in this document.



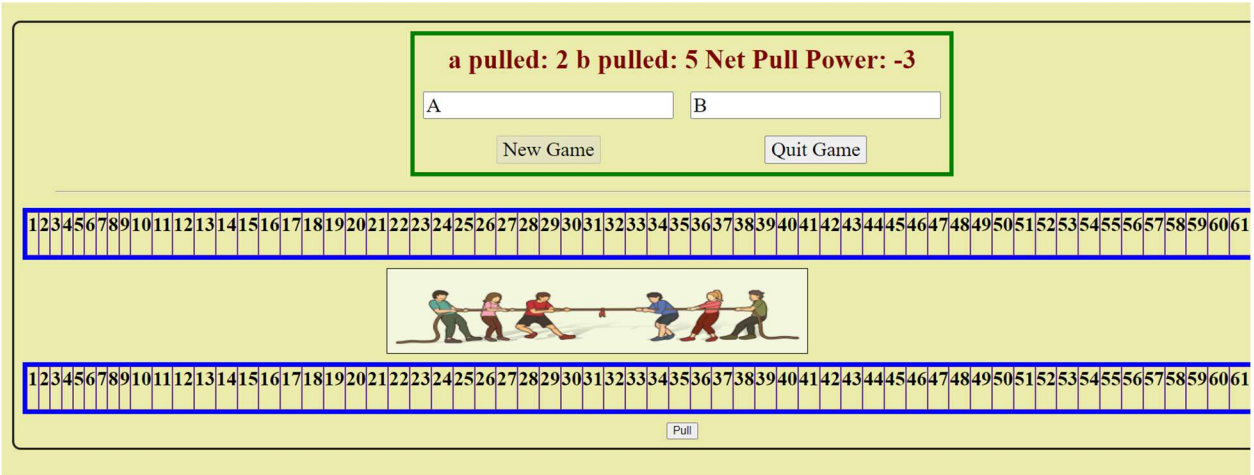
The game play itself is rather simple. We will provide a Pull button. When it is clicked, a new request will be sent to server. Server will generate 2 random numbers between 1 and 6 for each player. The player with the highest roll value will pull the other player the spaces which is the difference between the 2 rolls. For example, if Player 1 rolls a 5 and Player 2 rolls a 3, Player 1 pulls the other Player 2 spaces towards the pit which can be identified by a fixed position.

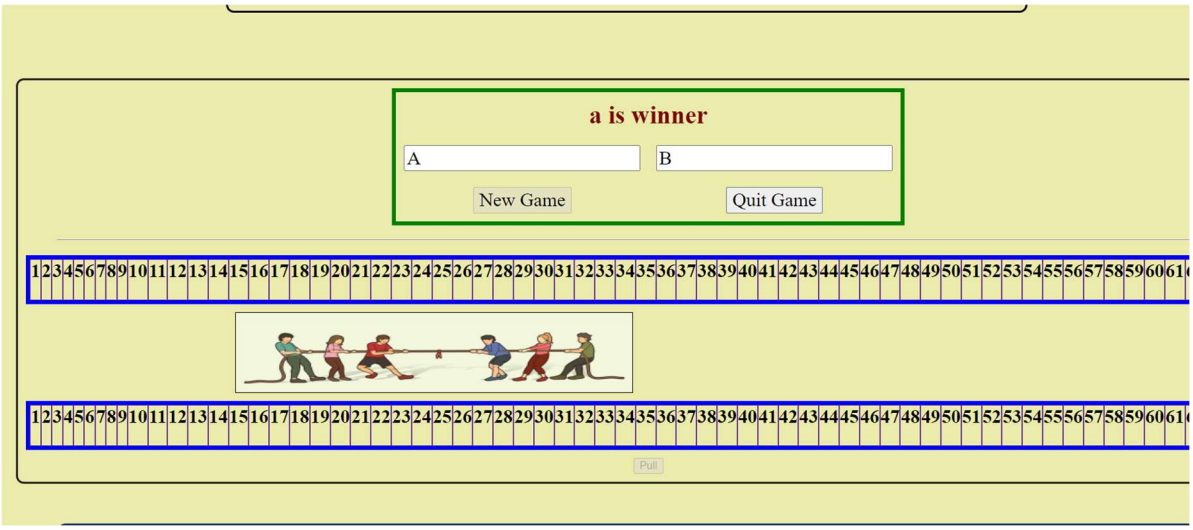
Need to used grid for the game board. In the example above a grid of 65 columns is taken and the Rope will always be 10 grid cells long. A winner is decided when the front of one Player is pulled into the pit or fixed position. If there is no winner after 30 pulls, then it is a draw and the round is over.

The server maintains ALL of the gameplay information. This means that the server “knows” the current state of the gameboard. Also remember that your intentions may be skirted by malicious users, so in all cases of data sent from client to server, the data shall be cleaned and validated (think data types, ranges, etc).

After the data has been validated and random numbers have been generated, you must check for a win condition. Again, a winner is decided when the front of one Player is pulled into the pit or fixed position [feel free to decide a fixed position]. If there is no winner after 30 pulls, then it is a draw and the round is over.

The following page shows a couple of possible game endings (note how the player names have persisted in the textboxes all along):


a pulled: 2 b pulled: 5 Net Pull Power: -3
A
B
New Game
Quit Game
Pull


a is winner
A
B
New Game
Quit Game
Pull

On the server side, each interaction should include interrogating what action has been requested by the client if action parameter is sent otherwise appropriate web service link will handle the interaction. Game initialization and new placement should be the actions you need.

For all actions, the game data received back on the client will need to be examined and the proper position of the rope will be determined by the values returned from the server.

As a best practice, you should set up a “return data” object and populate it while completing a required action. Return data back to the client where it may be processed in Javascript/jQuery. Remember that not just data but user messaging may be returned.

Note: This will be a living document as it is the first time it is being delivered. If typos are encountered, or an explanation is not clear or seems incomplete, please approach your instructor about it so that the feedback may be passed to the document creator. If the fix is complex enough, this document will be added to or changed and the class alerted. Do keep in mind that you are expected to do some of the thinking and specific design, so this document has not been designed as a step-by-step recipe for completing the assignment, but more intended as a discussion of the requirements.